

C++ 核心知识点易自学手册（第 5 章：循环和关系表达式）

前言

本章聚焦 C++ 中的循环结构（for、while、do-while）、关系表达式及相关操作，通过“概念解析 + 代码示例 + 对比分析 + 避坑指南”的形式，详细讲解循环的使用场景、控制逻辑及实战技巧，适合打印后系统学习。

5.1 for 循环

for 循环是**入口条件循环**（先判断条件，再执行循环体），适合已知循环次数的场景，结构清晰，将初始化、条件判断、更新操作集中在一处。

1. 基本语法与执行流程

（1）语法结构

```
for (初始化表达式; 测试表达式; 更新表达式) {  
    循环体; // 条件为true时执行  
}
```

（2）执行步骤

- 初始化**：执行一次初始化表达式（如 `int i=0`），通常用于设置循环起点。
- 测试条件**：判断测试表达式（如 `i<5`），若为 `true` 则执行循环体，否则退出循环。
- 执行循环体**：执行循环内的语句。
- 更新**：执行更新表达式（如 `i++`），修改循环变量，然后回到步骤 2 重复判断。

2. 关键特性与示例

(1) 循环变量的作用域

在 for 循环初始化部分声明的变量（如 `for(int i=0;...)`），仅在循环体内有效，出循环后失效：

```
for (int i=0; i<3; i++) {  
    cout << i << " "; // 输出：0 1 2  
}  
cout << i; // 错误！i在此处未定义
```

(2) 灵活的测试表达式

测试表达式可以是任意返回 `bool` 值的表达式，非 0 值视为 `true`，0 视为 `false`：

```
int limit = 3;  
for (int i=limit; i; i--) { // i为0时退出（0视为false）  
    cout << i << " "; // 输出：3 2 1  
}
```

(3) 使用循环处理数组

for 循环是遍历数组的常用工具，结合数组长度可高效访问元素：

```
int arr[5] = {10, 20, 30, 40, 50};  
for (int i=0; i<5; i++) {  
    cout << arr[i] << " "; // 输出：10 20 30 40 50  
}
```

3. 易错点与避坑

- **遗漏更新表达式**：导致循环变量不变，陷入死循环（如 `for(int i=0; i<5;)`）。
- **循环变量越界**：测试条件判断错误（如数组长度为 5，却用 `i<=5`），导致访问无效内存。

- **混淆初始化位置**：若循环变量在外部声明，需注意多次循环时的初始值是否正确：

```
int i;  
for (i=0; i<3; i++) { ... } // 正确  
for (i=0; i<2; i++) { ... } // 需重新初始化i=0，否则从3开始
```

5.2 while 循环

while 循环也是**入口条件循环**，但仅包含测试表达式，初始化和更新操作需在循环外或体内完成，适合循环次数不确定的场景。

1. 基本语法与执行流程

(1) 语法结构

```
初始化; // 循环前初始化变量  
while (测试表达式) {  
    循环体; // 条件为true时执行  
    更新; // 修改循环变量，避免死循环  
}
```

(2) 执行步骤

1. 先判断测试表达式，若为true则执行循环体，否则退出。
2. 循环体内需包含更新操作（如i++），否则会无限循环。

2. 示例与适用场景

(1) 遍历字符串（未知长度）

```
char str[] = "hello";  
int i = 0;  
while (str[i] != '\0') { // 遇到字符串结束符'\0'退出
```

```
cout << str[i]; // 输出: hello
i++;
}
```

(2) 与 for 循环的转换

for 循环可改写为 while 循环，二者功能等效：

```
// for循环
for (int i=0; i<3; i++) { cout << i; }
// 等效的while循环
int i=0;
while (i<3) {
    cout << i;
    i++;
}
```

3. 易错点与避坑

- **死循环**：忘记在循环体内更新循环变量（如 `while(i<5) { cout << i; }`）。
- **初始值错误**：循环前未正确初始化变量，导致测试条件一开始就为 `false`，循环体一次不执行。

5.3 do-while 循环

do-while 循环是**出口条件循环**（先执行循环体，再判断条件），确保循环体至少执行一次，适合需要“先执行后判断”的场景。

1. 基本语法与执行流程

(1) 语法结构

```
初始化;
do {
    循环体; // 至少执行一次
```

```
更新;  
} while (测试表达式); // 分号不能省略
```

(2) 执行步骤

1. 执行循环体（第一次无需判断条件）。
2. 执行更新操作。
3. 判断测试表达式，若为true则重复步骤 1，否则退出。

2. 示例与适用场景

(1) 获取用户输入（确保至少输入一次）

```
int num;  
do {  
    cout << "请输入1-10之间的数字：";  
    cin >> num;  
} while (num < 1 || num > 10); // 输入无效则重新输入
```

(2) 与 while 循环的区别

```
// do-while: 至少输出一次  
int a = 5;  
do {  
    cout << a; // 输出：5（即使条件为false）  
} while (a < 3);  
// while: 可能一次不执行  
int b = 5;  
while (b < 3) {  
    cout << b; // 无输出  
}
```

5.4 关系表达式与运算符

关系表达式用于比较两个值，返回bool类型（true或false），是循环条件的核心。

1. 关系运算符

运算符	含义	示例 (a=3, b=5)	结果
<	小于	a < b	true
<=	小于等于	a <= 3	true
==	等于	a == b	false
!=	不等于	a != b	true
>	大于	a > b	false
>=	大于等于	b >= 5	true

2. 易错点：赋值与相等判断

- **=是赋值运算符**：a = 5将5赋给a，表达式值为5（非0，视为true）。
- **==是相等运算符**：a == 5判断a是否等于5，返回true或false。

错误示例：

```
int x = 3;
if (x = 5) { // 错误！此处是赋值，表达式值为5（true），永远执行
    cout << "x是5";
}
```

正确示例：

```
if (x == 5) { // 正确：判断x是否为5
    ...
}
```

3. 递增 / 递减运算符

`++`（递增）和`--`（递减）用于变量加 1 或减 1，分前缀和后缀两种形式：

形式	含义	示例（a=3）	结果
前缀 <code>++a</code>	先加 1，再使用 a 的值	<code>b = ++a</code>	a=4, b=4
后缀 <code>a++</code>	先使用 a 的值，再加 1	<code>b = a++</code>	a=4, b=3

4. 组合赋值运算符

将算术运算与赋值结合，简化代码：

运算符	含义	等效表达式
<code>+=</code>	<code>a += b</code> $\rightarrow$ <code>a = a + b</code>	<code>x += 3</code> $\rightarrow$ <code>x = x + 3</code>
<code>-=</code>	<code>a -= b</code> $\rightarrow$ <code>a = a - b</code>	<code>y -= 2</code> $\rightarrow$ <code>y = y - 2</code>
<code>*=</code>	<code>a *= b</code> $\rightarrow$ <code>a = a * b</code>	<code>z *= 5</code> $\rightarrow$ <code>z = z * 5</code>

5.5 循环控制：复合语句与逗号运算符

循环体默认只能有一条语句，若需多条语句，需用**复合语句（代码块）**；逗号运算符可将多个表达式合并为一个，用于循环的初始化或更新部分。

1. 复合语句（代码块）

用`{}`将多条语句括起，视为一个整体：

```
for (int i=0; i<2; i++) {
    cout << "循环次数： " << i << endl;
    cout << "---" << endl; // 多条语句需用{}
}
```

2. 逗号运算符

将多个表达式分隔为一个，按顺序执行，整个表达式的值为最后一个表达式的值：

```
for (int i=0, j=5; i<j; i++, j--) { // 初始化和更新用逗号合并
    cout << i << " " << j << endl;
}
// 输出：0 5 → 1 4 → 2 3
```

5.6 文本输入与循环

循环常用来逐字符读取文本，需根据需求选择合适的输入方式（处理空格、换行符或 EOF）。

1. 三种输入方式对比

输入方式	特点	适用场景
<code>cin >> ch</code>	跳过空格、换行符、制表符	读取单词（忽略空白）
<code>cin.get(ch)</code>	读取所有字符（包括空白）	读取含空格的完整输入
<code>ch = cin.get()</code>	返回字符的 ASCII 码（int 类型），EOF 时返回EOF	兼容 C 语言风格输入

2. 示例：读取文本直到 EOF

```
// 使用cin.get(ch)读取所有字符，直到Ctrl+Z（Windows）或Ctrl+D（Unix）
char ch;
int count = 0;
while (cin.get(ch)) { // 读取失败（EOF）时退出
    cout << ch; // 回显字符
    count++;
}
cout << "\n共读取" << count << "个字符" << endl;
```

5.7 嵌套循环与二维数组

嵌套循环（循环内包含循环）适合处理二维数组（数组的数组），外层循环控制行，内层循环控制列。

1. 二维数组的声明与初始化

```
// 声明：3行4列的int数组（3个元素，每个元素是含4个int的数组）
int matrix[3][4] = {
    {1, 2, 3, 4}, // 第0行
    {5, 6, 7, 8}, // 第1行
    {9, 10, 11, 12} // 第2行
};
```

2. 嵌套循环遍历二维数组

```
// 外层循环遍历行，内层循环遍历列
for (int row=0; row<3; row++) {
    for (int col=0; col<4; col++) {
        cout << matrix[row][col] << " ";
    }
    cout << endl; // 每行结束换行
}
// 输出：
// 1 2 3 4
// 5 6 7 8
// 9 10 11 12
```

5.8 基于范围的 for 循环（C++11）

简化数组或容器的遍历，自动迭代所有元素，无需手动控制索引：

```
int arr[] = {10, 20, 30};
for (int x : arr) { // x依次表示arr的每个元素
    cout << x << " "; // 输出：10 20 30
}
// 修改元素需用引用（&）
```

```
for (int &x : arr) {  
    x *= 2; // 所有元素乘2  
}
```

重点警告与自查清单

1. **循环条件**：确保循环有明确的终止条件，避免死循环。
2. **变量作用域**：for 循环内声明的变量不可在循环外使用。
3. **赋值与相等**：永远用`==`判断相等，避免`if(x=5)`这类错误。
4. **数组越界**：循环访问数组时，索引不得超过长度-1。
5. **输入处理**：根据是否保留空白，选择`cin >> ch`或`cin.get(ch)`。

自测练习

1. 用 for 循环计算 1~100 的总和。
2. 用 while 循环读取用户输入的整数，直到输入 0，计算所有输入的平均值。
3. 用 do-while 循环实现一个简单的菜单（至少包含“退出”选项）。
4. 比较`i++`和`++i`在循环中的区别，用代码示例说明。
5. 定义一个 3 行 3 列的二维数组，用嵌套循环计算对角线元素的和。
6. 编写程序，用`cin.get(ch)`读取一行文本，统计其中空格的数量。

通过本章学习，需掌握三种循环的适用场景、关系表达式的正确使用，以及循环与数组、文本输入的结合技巧，为复杂程序设计打下基础。

（注：文档部分内容可能由 AI 生成）